

07 — Determinism

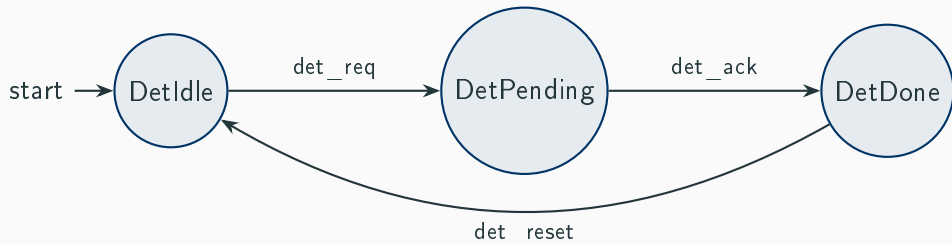
Deterministic vs Non-Deterministic Models

Mariano Cerrutti

Mununu Tutorial

Deterministic Protocol

In a **deterministic** automaton, each (state, label) pair has **exactly one** target:

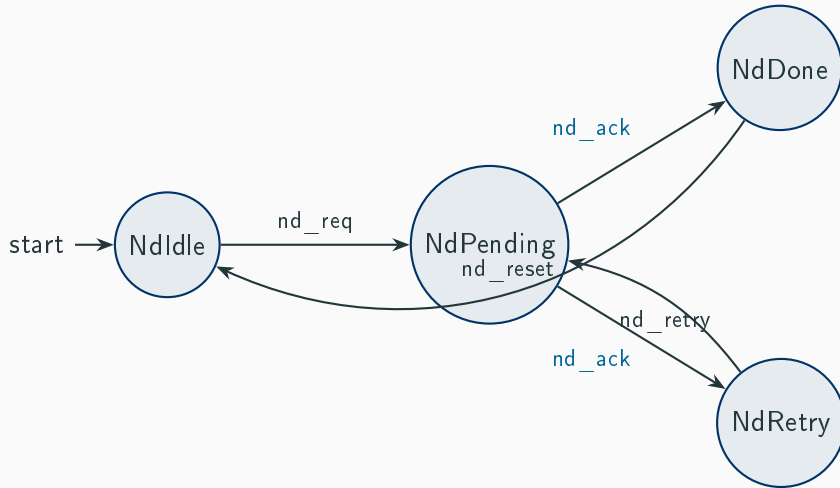


From **DetPending**, **det_ack** leads to *exactly one* state: **DetDone**.

No ambiguity — the next state is fully determined by the current state and the label.

Non-Deterministic Protocol

Now `nd_ack` from **NdPending** leads to *two different states*:



Impact on Verification: Box vs Diamond

Non-determinism changes the meaning of modalities:

□ — **Box (universal)**: “*All* successors satisfy φ ”

[labels = {nd_ack}] *NdDone* is **FALSE** for *NdPending*
because one nd_ack-successor is *NdRetry*, not *NdDone*.

◇ — **Diamond (existential)**: “*Some* successor satisfies φ ”

⟨labels = {nd_ack}⟩ *NdDone* is **TRUE** for *NdPending*
because one nd_ack-successor *is* *NdDone*.

In a **deterministic** model, box and diamond over a label agree (at most one successor).

Non-Deterministic Transitions in CTXDSL

```
1      }
2
3      transitions {
4          transition NdIdle -> NdPending on label nd_req;
5          // Non-deterministic choice: nd_ack leads to Done OR Retry
6          transition NdPending -> NdDone on label nd_ack;
7          transition NdPending -> NdRetry on label nd_ack;
8          // Retry loops back to try again
9          transition NdRetry -> NdPending on label nd_retry;
10         transition NdDone -> NdIdle on label nd_reset;
11     }
12 }
```

Lines 59–60: two transitions from **NdPending** on the *same* label `nd_ack`.

Formulas: Box vs Diamond

```
1 // Box modality: from NdPending, ALL nd_ack transitions lead to NdDone?
2 // This will be FALSE because nd_ack also leads to NdRetry
3 formula nd_ack_always_done {
4     over NondeterministicProtocol;
5     body = (! NdPending) || [ labels = { nd_ack } ] NdDone;
6 }
7
8 // Diamond modality: from NdPending, SOME nd_ack transition leads to
9 // NdDone?
10 // This will be TRUE (one branch does lead to NdDone)
11 formula nd_ack_some_done {
12     over NondeterministicProtocol;
13     body = (! NdPending) || < labels = { nd_ack } > NdDone;
14 }
```

- `nd_ack_always_done` uses `[ ]` (box) — requires *all* `nd_ack`-successors to be `NdDone`.
False for `NdPending`.

Takeaway

Concept	Meaning
Deterministic	Each (state, label) has at most one target. Fully predictable.
Non-deterministic	Same (state, label) can have multiple targets. Models uncertainty.
Box $[] \varphi$	Universal — <i>all</i> successors satisfy φ .
Diamond $\langle \rangle \varphi$	Existential — <i>some</i> successor satisfies φ .

Key insight: Non-determinism is not a bug — it models environments whose behavior we cannot predict. Choose box or diamond depending on whether you need *guarantees for all* or *existence of some* outcome.